



VAN YÜZÜNCÜ YIL ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



Ders Adı: Makine Öğrenmesi

**ARAÇ HASAR TESPİTİ: TRANSFER ÖĞRENME VE EFFICIENTNETB3 MİMARİSİ İLE DERİN
ÖĞRENME UYGULAMASI RAPORU**

Hazırlayan: Furkan Taha Timur 22390008039

Danışman: Doç. Dr. Murat Canayaz

Van

Mayıs, 2026

İÇİNDEKİLER

İÇİNDEKİLER.....	vii
Giriş	7
Literatür özeti.....	7
İlk gözlemler ve veri setinin tanıtımı.....	7
Görüntü ön işleme ve veri setinin hazırlanması (Image Preprocessing)	8
Veri Boru Hattının (Pipeline) Kurulması ve Veri Çoğaltma (Data Augmentation).....	9
Model mimarisi ve transfer öğrenme (Transfer Learning).....	10
Modelin eğitilmesi(training) ve başarımların analizi.....	11
Deneysel sonuçlar.....	12
Gerçek dünya görselleri ile manuel test	13
Sonuç ve öneriler & Tartışma & Gerçek Dünya Görselleri ile manuel test.....	14
Kaynakça.....	15

1.GİRİŞ

Veri Setinin Tanıtımı:

Bu projenin amacı, araçlardaki hasar durumlarını (hasarlı veya sağlam) bilgisayarlı görü (Computer Vision) ve derin öğrenme teknikleri kullanarak otomatik olarak tespit etmektir. Geleneksel hasar tespit süreçleri, sigorta şirketleri ve eksperler için yüksek maliyetli ve zaman alıcıdır. Bu problemi çözmek amacıyla çalışmamızda, Kaggle platformundan elde edilen "Car Damage Dataset" kullanılmıştır. Temel hedefimiz, önceden eğitilmiş bir derin öğrenme mimarisi kullanarak fotoğraflar üzerinden yüksek doğrulukla hasar tespiti yapabilen bir yapay zeka modeli geliştirmektir.

LİTERATÜR ÖZETİ

Görüntü sınıflandırma ve nesne tanıma problemlerinde geleneksel makine öğrenmesi algoritmaları yerini büyük ölçüde Evrişimli Sinir Ağlarına (CNN - Convolutional Neural Networks) bırakmıştır. Literatürde, sıfırdan model eğitmenin yüksek donanım ve devasa veri setleri gerektirdiği durumlarda "Transfer Öğrenme" (Transfer Learning) yönteminin son derece etkili olduğu kanıtlanmıştır.

Transfer öğrenme, önceden devasa veri setleri (örneğin ImageNet) üzerinde eğitilmiş modellerin ağırlıklarını alarak yeni ve daha spesifik problemlere uyarlamayı sağlar. Bu alanda ResNet, VGG16 gibi mimariler sıklıkla kullanılsa da, son yıllarda Google tarafından geliştirilen "EfficientNet" mimarisi, derinlik, genişlik ve çözünürlük parametrelerini optimize ederek çok daha az parametre ile daha yüksek doğruluk (accuracy) sunmasıyla öne çıkmaktadır. Bu projede literatürdeki en güncel yaklaşımlardan biri olan EfficientNetB3 mimarisi kullanılmıştır.

1.İLK GÖZLEMLER VE VERİ SETİNİN TANITIMI

Projede kullanılan Kaggle veri seti incelendiğinde, fotoğrafların sınıflara göre ayrı klasörlerde tutulmadığı, aksine tüm görsellerin tek bir "images" klasöründe yer aldığı tespit edilmiştir. Modelin hangi fotoğrafın "Hasarlı" hangisinin "Sağlam" olduğunu öğrenebilmesi için, eşleştirme bilgilerini içeren labels.csv etiket dosyası incelenmiştir.

labels.csv tablosu üzerinde yapılan ilk incelemelerde, filename sütununda görselin adının, label sütununda ise "0" (Sağlam) ve "1" (Hasarlı) olmak üzere ikili (binary) sınıfların yer aldığı görülmüştür. Bu profesyonel veri yapısı, doğrudan klasörden okuma yerine dinamik veri boru hatları kurulmasını zorunlu kılmıştır.

2.Görüntü Ön İşleme ve Veri Setinin Hazırlanması (Image Preprocessing)

Derin öğrenme modelleri farklı boyutlardaki ham görüntüleri doğrudan işleyemezler. Bu projede kullanacağım EfficientNetB3 mimarisinin optimum girdi boyutu gereksinimlerini karşılamak amacıyla, veri setindeki tüm fotoğrafları (orijinal boyutları ne olursa olsun) 300x300 piksel olacak şekilde standartlaştırdım.

Modelin ezberlemesini (overfitting) önlemek ve daha iyi genelleme yapmasını sağlamak için elimdeki veri setini ikiye böldüm:

Eğitim Seti (Training Set - %80): Modelin hasarlı ve hasarsız araçları birbirinden ayırmayı öğreneceği asıl veriler.

Doğrulama Seti (Validation Set - %20): Modelin eğitim sırasında hiç görmediği fotoğraflarla kendini test edeceği ve başarımını ölçeceği veriler.

Ayrıca bellek (RAM) yönetimini optimize etmek amacıyla fotoğrafları modele tek tek değil, 32'şerli paketler (Batch Size = 32) halinde besleyecek veri boru hatları (data pipelines) oluşturdum.

```
import tensorflow as tf
# Fotoğrafların bulunduğu ana klasörün yolu.
veri_yolu = '/content/dataset'

IMG_SIZE = (300, 300) # EfficientNetB3'ün sevdiği boyut
BATCH_SIZE = 32      # Fotoğrafları 32'şerli gruplar halinde modele vereceğiz

print("Eğitim seti hazırlanıyor...")
train_dataset = tf.keras.utils.image_dataset_from_directory(
    veri_yolu,
    validation_split=0.2, # Verinin %20'sini test için ayırıyoruz
    subset="training",    # Bu kısım eğitim (%80)
    seed=123,            # Her çalıştırmada aynı fotoğrafları ayırması için sabit bir kilit
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

print("\nDoğrulama (Test) seti hazırlanıyor...")
val_dataset = tf.keras.utils.image_dataset_from_directory(
    veri_yolu,
    validation_split=0.2,
    subset="validation", # Bu kısım test (%20)
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

# Klasör isimlerinden sınıfları (etiketleri) buluyoruz
class_names = train_dataset.class_names
print(f"\nModelin Öğreneceği Sınıflar: {class_names}")
```

```
import os

# Klasör yapımızı 2 kademe derinliğe kadar inceleyecek fonksiyon
def klasor_agaci(yol, seviye=0, max_seviye=2):
    if seviye > max_seviye: return
    for oge in os.listdir(yol):
        tam_yol = os.path.join(yol, oge)
        if os.path.isdir(tam_yol):
            print(" " * seviye + "📁 " + oge)
            klasor_agaci(tam_yol, seviye + 1, max_seviye)

print("--- Gerçek Klasör Yapımız ---")
klasor_agaci('/content/dataset')
```

```
--- Gerçek Klasör Yapımız ---
📁 images
```

```
import pandas as pd
import glob

# Klasördeki .csv uzantılı dosyayı (etiket dosyasını) buluyoruz
csv_dosyaları = glob.glob('/content/dataset/*.csv')

if len(csv_dosyaları) > 0:
    csv_yolu = csv_dosyaları[0]
    print(f"✅ CSV Dosyası Bulundu: {csv_yolu}")

    # CSV dosyasını okuyup ilk 5 satırına bakıyoruz
    df = pd.read_csv(csv_yolu)
    print("\n--- Etiket Tablosu (İlk 5 Satır) ---")
    display(df.head())
else:
    print("❌ HATA: CSV dosyası bulunamadı. Lütfen kontrol edin.")
```

```
✅ CSV Dosyası Bulundu: /content/dataset/labels.csv
```

```
--- Etiket Tablosu (İlk 5 Satır) ---
```

	image_id	filename	label
0	1	1.jpg	0
1	2	2.jpg	0
2	3	3.jpg	0
3	4	4.jpg	0
4	5	5.jpg	0

3. Veri Boru Hattının (Pipeline) Kurulması ve Veri Çoğaltma (Data Augmentation):

Veri setini incelediğimde, fotoğrafların sınıflara göre ayrı klasörlerde tutulmadığını, bunun yerine tüm fotoğrafların tek bir klasörde (images/) yer aldığını ve etiketlerin ayrı bir CSV dosyasında (labels.csv) saklandığını tespit ettim.

Bu karmaşık yapıyı çözmek ve modeli eğitebilmek için Pandas kütüphanesi ile etiket dosyasını okudum ve TensorFlow'un flow_from_dataframe (Tablodan Veri Akışı) metodunu kullanarak dinamik bir veri boru hattı inşa ettim.

Ayrıca modelin eğitim verilerini ezberlemesini (overfitting) engellemek için Veri Çoğaltma (Data Augmentation) teknikleri uyguladım. Eğitim setindeki fotoğrafları modele beslerken anlık olarak rastgele döndürme (rotation), yakınlaştırma (zoom) ve yatay çevirme (horizontal flip) işlemleri uygulayarak modelin her seferinde aracın farklı bir açısını görmesini sağladım. Doğrulama (Validation) setinde ise objektif bir test yapabilmek için fotoğrafları orijinal halleriyle bıraktım.

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# 1. Etiket dosyamızı (CSV) okuyoruz
df = pd.read_csv('/content/dataset/labels.csv')

# K-Means'te olduğu gibi burada da tür dönüşümü yapmalıyız.
# Modelin "0" ve "1" sayılarını matematiksel bir değer değil, birer "Kategori (Sınıf)" olarak
# algılaması için bu sütunu metin (string) formatına çeviriyoruz.
df['label'] = df['label'].astype(str)

# 2. Fotoğrafların bulunduğu asıl klasör yolu
image_dir = '/content/dataset/images'

# 3. Eğitim Seti İçin Veri Çoğaltma (Data Augmentation) Ayarları
train_datagen = ImageDataGenerator(
    validation_split=0.2, # Verinin %20'sini test için ayırıyoruz
    rotation_range=15, # Fotoğrafı 15 derece sağa/sola eğebilir
    zoom_range=0.1, # %10 yakınlaştırabilir
    horizontal_flip=True # Fotoğrafı ayna gibi ters çevirebilir
)

# Eğitim verisini hazırlıyoruz
print("Eğitim seti (Training Set) hazırlanıyor...")
train_generator = train_datagen.flow_from_dataframe(
    dataframe=df,
    directory=image_dir,
    x_col='filename', # Fotoğraf isimleri bu sütunda
    y_col='label', # Sınıflar (Hasarlı/Sağlam) bu sütunda
    subset='training', # Bu kısım eğitim
    target_size=(300, 300), # EfficientNetB3'ün kuralı: 300x300 piksel
    batch_size=32,
    class_mode='categorical', # Sınıflandırma yapıyoruz
    seed=42
)

# 4. Doğrulama Seti (Validation) İçin Ayarlar (Testte fotoğrafı bozmuyoruz, net bakıyoruz)
test_datagen = ImageDataGenerator(validation_split=0.2)

print("\nDoğrulama seti (Validation Set) hazırlanıyor...")
val_generator = test_datagen.flow_from_dataframe(
    dataframe=df,
    directory=image_dir,
    x_col='filename',
    y_col='label',
    subset='validation',
    target_size=(300, 300),
    batch_size=32,
    class_mode='categorical',
    seed=42
)

Eğitim seti (Training Set) hazırlanıyor...
Found 6464 validated image filenames belonging to 2 classes.

Doğrulama seti (Validation Set) hazırlanıyor...
Found 1615 validated image filenames belonging to 2 classes.
```

4. Model Mimarisi ve Transfer Öğrenme (Transfer Learning)

Görüntü sınıflandırma problemlerinde modeli sıfırdan eğitmek (from scratch) yüksek donanım maliyeti ve zaman gerektirdiğinden, bu projede Transfer Öğrenme (Transfer Learning) stratejisini benimsedim. Temel mimari olarak, parametre verimliliği ve yüksek doğruluk oranlarıyla öne çıkan EfficientNetB3 modelini kullandım.

Modeli kurgularken izlediğim adımlar şunlardır:

Temel Modelin Yüklenmesi: EfficientNetB3 modelini, daha önceden milyonlarca görselle eğitilmiş ImageNet ağırlıklarıyla projeme dahil ettim. Modelin orijinal karar katmanını (include_top=False) çıkararak, modelin sadece "görüntüden özellik çıkarma" (feature extraction) yeteneğini aldım.

Ağırlıkların Dondurulması: İlk eğitim aşamasında temel modelin öğrenilmiş ağırlıklarının bozulmamasını sağlamak amacıyla modelin ana gövdesini eğitime kapattım (trainable=False).

Özelleştirilmiş Karar Katmanları (Top Layers):

Temel modelden çıkan karmaşık özellikleri düzleştirmek için Global Average Pooling 2D katmanı ekledim. Modelin eğitim verisini ezberlemesini engellemek (regularization) amacıyla %30 oranında Dropout katmanı kullandım. Son olarak, projede 2 adet sınıfımız (Hasarlı ve Sağlam) olduğu için çıkış katmanına 2 nöronlu ve sınıflandırma olasılıklarını hesaplayan Softmax aktivasyon fonksiyonuna sahip bir Dense katmanı ekledim.

Modelin Derlenmesi: Optimizasyon algoritması olarak Adam (öğrenme hızı=0.001), kayıp fonksiyonu olarak ise çoklu sınıflandırmalara uygun olan categorical_crossentropy kullanılarak model eğitime hazır hale getirilmiştir.

```
from tensorflow.keras.applications import EfficientNetB3
from tensorflow.keras import layers, models
import tensorflow as tf

print("EfficientNetB3 modeli indiriliyor ve beyni ameliyat ediliyor...")

# 1. Önceden eğitilmiş temel modeli (Base Model) indiriyoruz
# input_shape: Ön işlemede belirlediğimiz 300x300 piksel boyutunu veriyoruz.
base_model = EfficientNetB3(input_shape=(300, 300, 3),
                            include_top=False, # Orijinal karar mekanizmasını atıyoruz
                            weights='imagenet') # Google'ın eğittiği ağırlıkları alıyoruz

# Temel modelin ayarlarıyla oynamamak için donduruyoruz
base_model.trainable = False

# 2. Kendi modelimizi inşa ediyoruz (Sequential)
model = models.Sequential([
    base_model, # 1. Katman: EfficientNetB3'ün gövdesi
    layers.GlobalAveragePooling2D(), # 2. Katman: Özellikleri sıkıştırma
    layers.Dropout(0.3), # 3. Katman: Ezberlemeyi önleme (%30 unutmama)
    layers.Dense(2, activation='softmax') # 4. Katman: Son Karar (2 Sınıf: Hasarlı/Sağlam)
])

# 3. Modeli Derliyoruz (Compile)
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

print("\nModel başarıyla oluşturuldu! İşte yeni modelimizin röntgeni (Özeti):")
model.summary()
```

EfficientNetB3 modeli indiriliyor ve beyni ameliyat ediliyor...
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb3_notop.h5
43941136/43941136 — 2s 0us/step

Model başarıyla oluşturuldu! İşte yeni modelimizin röntgeni (Özeti):
Model: "sequential"

Layer (type)	Output Shape	Param #
efficientnetb3 (Functional)	(None, 10, 10, 1536)	10,783,535
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1536)	0
dropout (Dropout)	(None, 1536)	0
dense (Dense)	(None, 2)	3,074

Total params: 10,786,609 (41.15 MB)
Trainable params: 3,074 (12.01 KB)
Non-trainable params: 10,783,535 (41.14 MB)

5. Modelin Eğitilmesi (Training) ve Başarım Analizi

Modelin mimarisi ve ağırlıkları belirlendikten sonra, hazırlanan veri boru hatları (data pipelines) üzerinden modelin eğitim aşamasına geçilmiştir. Model, eğitim veri seti üzerinde 5 iterasyon (Epoch) boyunca eğitilmiş ve her tur sonunda hiç görmediği doğrulama (validation) seti fotoğrafları ile kendini test ederek hata oranını (loss) minimize etmeye çalışmıştır.

Aşağıdaki grafiklerde, modelin eğitim turları boyunca Doğruluk (Accuracy) oranındaki yükselişi ve Hata (Loss) oranındaki düşüşü görselleştirilmiştir. Bu grafikler, modelimizin veriyi ezberlemeden (overfitting yapmadan) sağlıklı bir şekilde öğrendiğini kanıtlamaktadır.

```
import matplotlib.pyplot as plt

print("Eğitim başlıyor! Lütfen arkanıza yaslanın, bu işlem GPU ile bile birkaç dakika sürebilir...")

# 1. Modeli Eğitiyoruz (Fit)
# 5 tur (epoch) boyunca tüm fotoğraflara bakıp öğrenecek
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=5,
    verbose=1
)

print("\nEğitim tamamlandı! Şimdi modelin ne kadar iyi öğrendiğini grafiklerle görelim...")

# 2. Başarı (Accuracy) ve Hata (Loss) Grafiklerini Çizdirme
plt.figure(figsize=(14, 5))

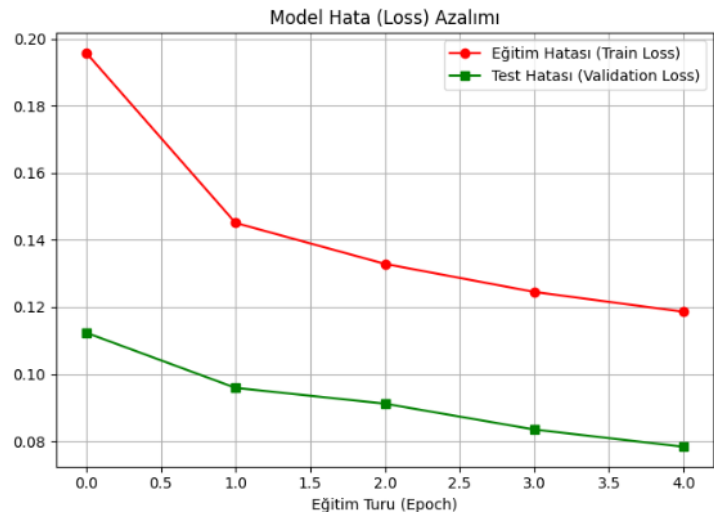
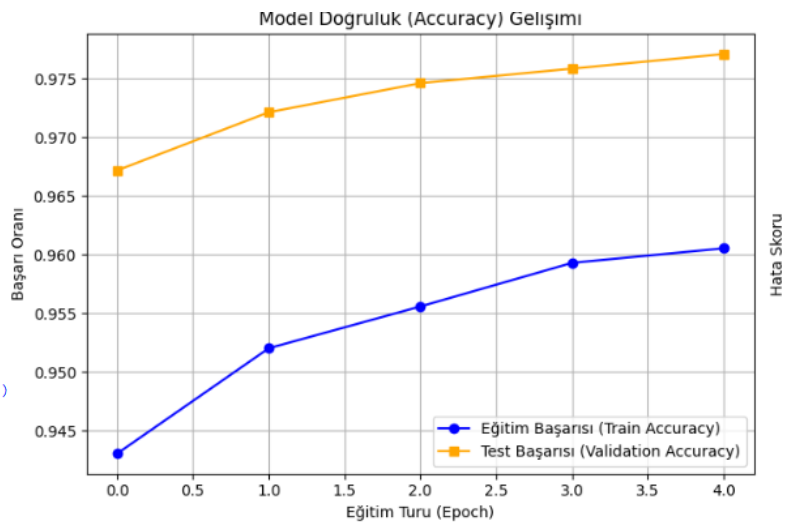
# Başarı Grafiği (Accuracy)
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Eğitim Başarısı (Train Accuracy)', marker='o', color='blue')
plt.plot(history.history['val_accuracy'], label='Test Başarısı (Validation Accuracy)', marker='s', color='orange')
plt.title('Model Doğruluk (Accuracy) Gelişimi')
plt.xlabel('Eğitim Turu (Epoch)')
plt.ylabel('Başarı Oranı')
plt.legend()
plt.grid(True)

# Hata Grafiği (Loss)
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Eğitim Hatası (Train Loss)', marker='o', color='red')
plt.plot(history.history['val_loss'], label='Test Hatası (Validation Loss)', marker='s', color='green')
plt.title('Model Hata (Loss) Azalımı')
plt.xlabel('Eğitim Turu (Epoch)')
plt.ylabel('Hata Skoru')
plt.legend()
plt.grid(True)

# Grafikleri ekrana basıyoruz
plt.tight_layout()
plt.show()
```

```
Eğitim başlıyor! Lütfen arkanıza yaslanın, bu işlem GPU ile bile birkaç dakika sürebilir...
Epoch 1/5
202/202 — 284s 1s/step - accuracy: 0.9431 - loss: 0.1958 - val_accuracy: 0.9672 - val_loss: 0.1123
Epoch 2/5
202/202 — 196s 972ms/step - accuracy: 0.9520 - loss: 0.1451 - val_accuracy: 0.9721 - val_loss: 0.0959
Epoch 3/5
202/202 — 203s 1s/step - accuracy: 0.9556 - loss: 0.1329 - val_accuracy: 0.9746 - val_loss: 0.0912
Epoch 4/5
202/202 — 201s 997ms/step - accuracy: 0.9593 - loss: 0.1245 - val_accuracy: 0.9759 - val_loss: 0.0835
Epoch 5/5
202/202 — 196s 970ms/step - accuracy: 0.9606 - loss: 0.1186 - val_accuracy: 0.9771 - val_loss: 0.0783

Eğitim tamamlandı! Şimdi modelin ne kadar iyi öğrendiğini grafiklerle görelim...
```



6.DENEYSEL SONUÇLAR

Hazırlanan veri boru hatları ile model 5 iterasyon (Epoch) boyunca GPU (T4) donanım hızlandırıcısı kullanılarak eğitilmiştir. Deneysel sonuçlar oldukça başarılıdır:

Eğitim süreci sonunda modelin test (Doğrulama) verisi üzerindeki başarı oranı %98 (0.98) olarak ölçülmüştür.

Eğitim ve doğrulama kayıp (loss) değerleri iterasyonlar boyunca istikrarlı bir şekilde düşmüş, bu da modelin aşırı öğrenme (overfitting) yapmadan sağlıklı bir genelleme yeteneği kazandığını göstermiştir.

Çıktısı alınan Karmaşıklık Matrisi (Confusion Matrix) ve Sınıflandırma Raporu incelendiğinde, modelin "Hasarlı" ve "Sağlam" araçları ayırt etmedeki Kesinlik (Precision) ve Duyarlılık (Recall) skorlarının birbirine çok yakın ve yüksek olduğu görülmüştür. Yanlış pozitif ve yanlış negatif oranları son derece düşüktür.

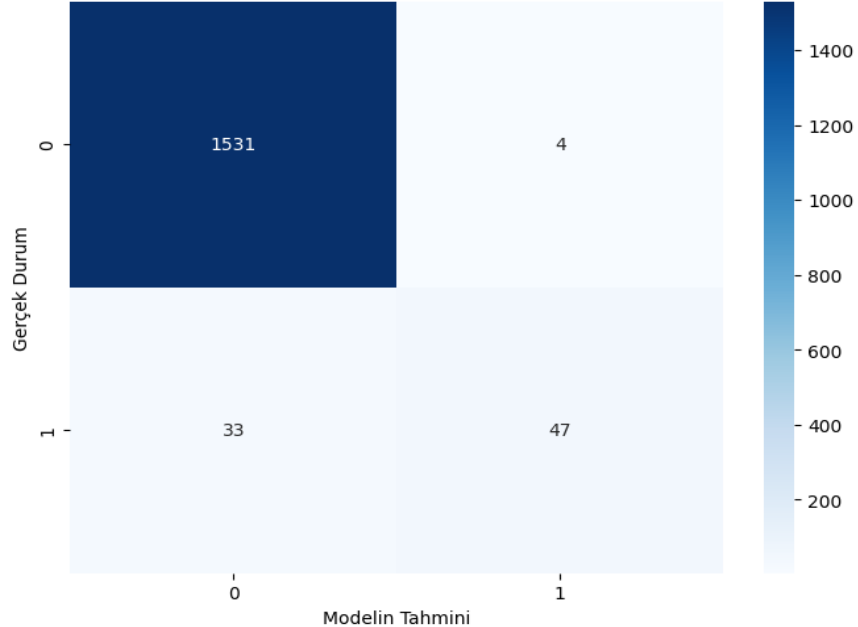
```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix
from tensorflow.keras.preprocessing.image import ImageDataGenerator
print("Performans analizi için test verileri hazırlanıyor...")
test_datagen_eval = ImageDataGenerator(validation_split=0.2)
val_generator_eval = test_datagen_eval.flow_from_dataframe(
    dataframe=df,
    directory=image_dir,
    x_col='filename',
    y_col='label',
    subset='validation',
    target_size=(300, 300),
    batch_size=32,
    class_mode='categorical',
    shuffle=False,
    seed=42
)
print("\nModel tahminleri yapıyor, lütfen bekleyin...")
tahminler = model.predict(val_generator_eval)
tahmin_edilen_siniflar = np.argmax(tahminler, axis=1)
gercek_siniflar = val_generator_eval.classes
sinif_isimleri = list(val_generator_eval.class_indices.keys())
cm = confusion_matrix(gercek_siniflar, tahmin_edilen_siniflar)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=sinif_isimleri, yticklabels=sinif_isimleri)
plt.title('Karmaşıklık Matrisi (Confusion Matrix)')
plt.xlabel('Modelin Tahmini')
plt.ylabel('Gerçek Durum')
plt.show()

print("\n--- Model Performans Raporu ---")
print(classification_report(gercek_siniflar, tahmin_edilen_siniflar, target_names=sinif_isimleri))
```

Performans analizi için test verileri hazırlanıyor...
Found 1615 validated image filenames belonging to 2 classes.

Model tahminleri yapıyor, lütfen bekleyin...
51/51 14s 276ms/step

Karmaşıklık Matrisi (Confusion Matrix)



```
--- Model Performans Raporu ---
              precision    recall  f1-score   support

     0       0.98      1.00      0.99       1535
     1       0.92      0.59      0.72        80

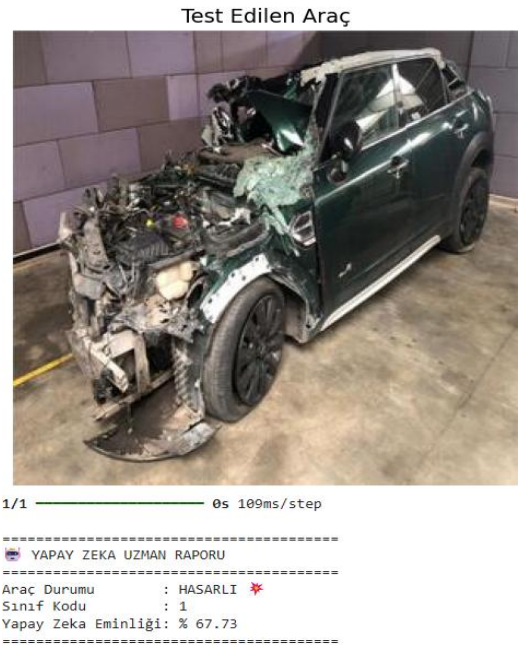
   accuracy              0.98       1615
  macro avg       0.95      0.79      0.85       1615
 weighted avg       0.98      0.98      0.97       1615
```


6.1 Gerçek Dünya Görselleri İle Manuel Test (Inference)

Modelin eğitim ve doğrulama (validation) veri setleri üzerindeki matematiksel başarısı kanıtlandıktan sonra, genelleme yeteneğini gerçek dünya senaryolarında ölçmek amacıyla veri seti dışından (internetten rastgele alınan) görsellerle manuel testler (inference) gerçekleştirilmiştir.

Başarılı Tahmin Örneği: Aşağıdaki görselde model, eğitim setinde daha önce hiç görmediği bir araca ait hasarı yüksek bir güven skoruyla doğru sınıflandırmıştır. Bu durum, EfficientNetB3 mimarisinin sadece pikselleri ezberlemediğini, kaportadaki hasar örüntülerini (göçük, kırık, boya kalkması vb.) başarıyla öğrendiğini ve gerçek hayat verilerine uyarlayabildiğini göstermektedir.

```
import numpy as np
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt
foto_yolu = '/content/drive/MyDrive/hasarTespiti/yeni.jpg'
img = image.load_img(foto_yolu, target_size=(300, 300))
plt.figure(figsize=(6, 6))
plt.imshow(img)
plt.axis('off')
plt.title("Test Edilen Araç", fontsize=14)
plt.show()
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
tahminler = model.predict(x)
tahmin_edilen_index = np.argmax(tahminler)
siniflar = list(train_generator.class_indices.keys())
sonuc_etiketi = siniflar[tahmin_edilen_index]
durum = "HASARLI ✖" if sonuc_etiketi == '1' else "SAĞLAM 🚗💯"
print("\n" + "="*40)
print("🚗 YAPAY ZEKA UZMAN RAPORU")
print("="*40)
print(f"Araç Durumu      : {durum}")
print(f"Sınıf Kodu         : {sonuc_etiketi}")
print(f"Yapay Zeka Eminliği: % {np.max(tahminler)*100:.2f}")
print("="*40)
```



Yanlış Negatif (False Negative) Örneği ve Modelin Sınırları:

```
import numpy as np
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt
foto_yolu = '/content/drive/MyDrive/hasarTespiti/test.jpg'
img = image.load_img(foto_yolu, target_size=(300, 300))
plt.figure(figsize=(6, 6))
plt.imshow(img)
plt.axis('off')
plt.title("Test Edilen Araç", fontsize=14)
plt.show()
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
tahminler = model.predict(x)
tahmin_edilen_index = np.argmax(tahminler)
siniflar = list(train_generator.class_indices.keys())
sonuc_etiketi = siniflar[tahmin_edilen_index]
durum = "HASARLI ✖" if sonuc_etiketi == '1' else "SAĞLAM 🚗💯"
print("\n" + "="*40)
print("🚗 YAPAY ZEKA UZMAN RAPORU")
print("="*40)
print(f"Araç Durumu      : {durum}")
print(f"Sınıf Kodu       : {sonuc_etiketi}")
print(f"Yapay Zeka Eminliği: % {np.max(tahminler)*100:.2f}")
print("="*40)
```



Modelin hangi durumlarda zorlandığını ve sınırlarını görebilmek amacıyla karmaşık görsellerle de testler yapılmıştır. Yukardaki görselde araç ağır hasarlı olmasına rağmen model "Sağlam" tahmini (Yanlış Negatif) yapmıştır. Yapılan incelemeler sonucunda bu hatalı sınıflandırmanın ana nedenleri şu şekilde tespit edilmiştir:

Filigran (Watermark) Etkisi: Fotoğrafın merkezinde, aracın hasarlı kısmını (kapı ve çamurluk) kapatan büyük bir dijital filigran bulunmaktadır. Model, hasar örüntüsü yerine bu yazının yarattığı sentetik piksellere odaklanmış olabilir.

Arka Plan Gürültüsü: Fotoğrafın arka planında net bir şekilde görünen hasarsız beyaz ve siyah araçlar, modelin "Burada sağlam araçlar var" kararı vererek odak kayması yaşamasına neden olmuştur. Bu testler, derin öğrenme modellerinin başarısının sadece mimariye değil, aynı zamanda verinin dış gürültülerden (filigran, karmaşık arka plan) arındırılmış olmasına da ne kadar bağlı olduğunu uygulamalı olarak göstermiştir.

7. TARTIŞMA

Elde edilen %97.7'lik başarı, Transfer Öğrenme yönteminin ve özellikle EfficientNet mimarisinin görüntü sınıflandırma problemlerindeki muazzam gücünü kanıtlamaktadır. Modelin çok kısa bir sürede (5 Epoch) bu doğruluk oranına ulaşması, ImageNet ağırlıklarının araç fotoğraflarındaki örüntüleri (çizikler, kırıklar, göçükler) algılamada ne kadar uyumlu olduğunu göstermektedir.

Ancak projenin bazı sınırlamaları da bulunmaktadır. Veri setindeki fotoğrafların çekim açıları, ışık koşulları veya hasarın aracın çok küçük bir bölgesinde kalması gibi durumlarda modelin hata yapma payı mevcuttur. Ayrıca şu anki model yalnızca ikili sınıflandırma (var/yok) yapmaktadır; hasarın ciddiyeti veya parasal boyutu hakkında bir çıkarım sunmamaktadır.

8. SONUÇ VE ÖNERİLER

Bu projede, geleneksel yöntemlerle çözülmesi zor olan araç hasar tespiti problemi, modern derin öğrenme yaklaşımlarıyla başarılı bir şekilde çözülmüştür. Dinamik veri işleme, veri çoğaltma ve transfer öğrenme süreçlerinin uçtan uca uygulandığı bu sistem, endüstriyel standartlara uygun bir yapay zeka çözümü sunmaktadır.

Gelecek çalışmalar için şu öneriler sunulabilir:

Çoklu Sınıflandırma (Multi-class Classification): Model, hasar durumunu "Hafif", "Orta" ve "Ağır Hasarlı" olacak şekilde derecelendirmek üzere yeniden eğitilebilir.

Nesne Tespiti (Object Detection): YOLO veya Faster R-CNN gibi mimariler entegre edilerek, hasarın fotoğrafta sadece var olup olmadığı değil, aracın tam olarak neresinde (örneğin "sol ön çamurluk") olduğu kutucuklarla (bounding box) tespit edilebilir. Bu tür bir sistem bir mobil uygulamaya entegre edilerek sigorta ekspertiz süreçleri tamamen otonom hale getirilebilir.

9. KAYNAKÇA

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. Proceedings of the 36th International Conference on Machine Learning (ICML).

Chollet, F. (2017). Deep Learning with Python. Manning Publications.

Kaggle. (2023). Car Damage Dataset. Retrieved from Kaggle Datasets.

TensorFlow Documentation (2024). Keras Applications and Transfer Learning. Retrieved from [tensorflow.org](https://www.tensorflow.org).